

AI QUALITY

O Desafio Gilded Rose

Testes unitários gerados por IA sob auditoria: 5 técnicas de prompt, um kata clássico e uma lacuna que 100% de cobertura não revelou.

Go

ChatGPT

Gilded Rose Kata

Arthur Lara Panzera • Albert Luis • Diogo Caribe • Felipe Augusto Pereira • Gabriel Reis Lebron • Gustavo Azi Prehl Gama

Setup do experimento

Objetivo: investigar se diferentes técnicas de prompt engineering mudam a qualidade dos testes unitários gerados por IA, e auditar criticamente se cobrem de fato as regras de negócio do kata, não apenas o código como ele foi escrito.

Configuração

Linguagem	Go
LLM utilizada	ChatGPT
Kata base	Gilded Rose (Emily Bache)
Repositório	https://github.com/gustavoprehl/TesteDeSoftware

Estrutura da equipe

Dupla 1
Engenharia de Prompt
Testar e documentar as 5 técnicas

Dupla 2
Implementação em Go
Integrar testes, corrigir erros, capturar evidências

Dupla 3
Auditoria & Análise
Auditar cobertura, escrever veredito, montar slides

O código legado - gildedrose.go

- **Função central:** UpdateQuality(items []Item), iterando sobre []Item
- **Struct:** Item { Name string; SellIn, Quality int }
- **Tipos tratados:** Normal, Aged Brie, Sulfuras, Backstage Passes
- **Regras de limite:** Quality nunca < 0 nem > 50 (exceto Sulfuras, fixo)

Ponto crítico (revelado na auditoria)

O fixture de demonstração do kata cita “Conjured Mana Cake”, mas o código não implementa a regra de degradação dupla, não há nenhum if para esse tipo.

```
cd: no such file or directory: gildedrose.go
>>> 1/6 - Código legado (gildedrose/gildedrose.go) - a logica condicional confusa que a IA teve que testar. (sem verde/vermelho aqui - e so o contexto)
Pressione Enter para continuar...
package gildedrose

type Item struct {
    Name      string
    SellIn, Quality int
}

func UpdateQuality(items []Item) {
    for i := 0; i < len(items); i++ {
        if items[i].Name != "Aged Brie" {
            if items[i].Quality > 0 {
                if items[i].Name != "Sulfuras, Hand of Ragnaros" {
                    items[i].Quality = items[i].Quality - 1
                }
            }
        } else {
            if items[i].Quality < 50 {
                items[i].Quality = items[i].Quality + 1
                if items[i].Name == "Backstage passes to a TAFKAL80ETC concert" {
                    if items[i].SellIn < 11 {
                        if items[i].Quality < 50 {
                            items[i].Quality = items[i].Quality + 1
                        }
                    }
                }
                if items[i].SellIn < 6 {
                    if items[i].Quality < 50 {
                        items[i].Quality = items[i].Quality + 1
                    }
                }
            }
        }
        if items[i].Name != "Sulfuras, Hand of Ragnaros" {
            items[i].SellIn = items[i].SellIn - 1
        }
        if items[i].SellIn < 0 {
            if items[i].Name != "Aged Brie" {
                if items[i].Name != "Backstage passes to a TAFKAL80ETC concert" {
                    if items[i].Quality > 0 {
                        if items[i].Name != "Sulfuras, Hand of Ragnaros" {
                            items[i].Quality = items[i].Quality - 1
                        }
                    }
                } else {
                    items[i].Quality = items[i].Quality - items[i].Quality
                }
            } else {
                if items[i].Quality < 50 {
                    items[i].Quality = items[i].Quality + 1
                }
            }
        }
    }
}
```

Metodologia - 5 técnicas de prompt testadas

1

Direto

Colar o código e pedir a suíte diretamente

13 testes

2

Chain-of-Thought

Raciocínio passo a passo antes do código

6 funções + subtestes

3

Persona Pattern

Papel de Senior QA Engineer, 15 anos

8 funções + subtestes

4

Few-shot

Template AAA fornecido como exemplo

24 testes

5

Spec-driven (bônus)

Regras de negócio explícitas no preâmbulo

23 testes

Resultados comparativos e ranking

Técnica	Nº testes	Compila?	Cobertura de borda	Ranking
Persona Pattern	8 (+subtestes)	Sim	Mais completa; único com nil/vazio	1º
Chain-of-Thought	6 (+subtestes)	Sim	Boa; amplia contrato de Sulfuras	2º
Spec-driven	23	Sim	Boa; metadado incorreto sobre falhas	3º
Direto (baseline)	13	Sim	Superficial - só valores no limite	4º
Few-shot	24	Não → corrigido	Boa, após correção estrutural	5º

```
PASS prompt_tests/chain_of_thought.TestSulfuras/does_not_change (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras/does_not_change_with_SellIn_zero (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras/does_not_change_after_expiration (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras/quality_above_50_is_preserved (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras (0.00s)
PASS prompt_tests/chain_of_thought.TestBackstagePasses/more_than_10_days_remaining_increases_quality_by_1 (0.00s)
```

Estudo de caso: Few-shot - de RED a GREEN

Erro de sintaxe



testing.t minúsculo não existe em Go (linha 140)

Erro estrutural



gr := GildedRose{Items: items}
- struct inexistente no código real (11 erros “undefined”)

Corrigido



24 PASS após as duas correções, nenhuma asserção alterada

```
0 tests, 1 failure, 1 error in 0.000s
>>> 3/6 - RED #2: mesmo corrigindo o typo, ainda nao compila -> 'undefined: GildedRose'. A IA usou uma struct do exemplo do prompt que nao existe no codigo real (a assinatura real e UpdateQuality(items []*Item)),
Pressione Enter para continuar...
FAIL _evidence/few_shot_typo_fixed_only

=== Failed
=== FAIL: _evidence/few_shot_typo_fixed_only (0.00s)
FAIL github.com/emilybache/gildedrose-refactoring-kata/_evidence/few_shot_typo_fixed_only [build failed]

=== Errors
_evidence/few_shot_typo_fixed_only/few_shot_test.go:17:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:31:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:45:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:59:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:73:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:87:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:101:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:115:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:129:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:143:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:143:8: too many errors

DONE 0 tests, 1 failure, 11 errors in 0.000s
```

Evidências consolidadas

131

testes executados

0

FAIL

100%

cobertura de statement

```
>> 5/6 - GREEN: as 5 tecnicas de prompt rodando juntas. 131 PASS, 0 FAIL
Pressione Enter para continuar...
PASS prompt_tests/chain_of_thought.TestNormalItem/normal_item_before_expiration (0.00s)
PASS prompt_tests/chain_of_thought.TestNormalItem/normal_item_with_SellIn_zero (0.00s)
PASS prompt_tests/chain_of_thought.TestNormalItem/normal_item_after_expiration (0.00s)
PASS prompt_tests/chain_of_thought.TestNormalItem/normal_item_with_zero_quality (0.00s)
PASS prompt_tests/chain_of_thought.TestNormalItem/normal_item_with_quality_50 (0.00s)
PASS prompt_tests/chain_of_thought.TestNormalItem/unknown_item_is_treated_as_normal (0.00s)
PASS prompt_tests/chain_of_thought.TestNormalItem (0.00s)
PASS prompt_tests/chain_of_thought.TestAgedBrie/increases_quality_before_expiration (0.00s)
PASS prompt_tests/chain_of_thought.TestAgedBrie/SellIn_zero_gets_normal_increase_and_expiration_increase (0.00s)
PASS prompt_tests/chain_of_thought.TestAgedBrie/increases_quality_after_expiration (0.00s)
PASS prompt_tests/chain_of_thought.TestAgedBrie/quality_zero (0.00s)
PASS prompt_tests/chain_of_thought.TestAgedBrie/quality_49_reaches_maximum (0.00s)
PASS prompt_tests/chain_of_thought.TestAgedBrie/quality_50_stays_at_maximum (0.00s)
PASS prompt_tests/chain_of_thought.TestAgedBrie (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras/does_not_change (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras/does_not_change_with_SellIn_zero (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras/does_not_change_after_expiration (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras/quality_above_50_is_preserved (0.00s)
PASS prompt_tests/chain_of_thought.TestSulfuras (0.00s)
```

```
>>> 6/6 - Cobertura de statement do pacote gildedrose, por tecnica (nao e verde/vermelho de teste - e metrica)
Pressione Enter para continuar...
direto          coverage: 100.0%
chain_of_thought coverage: 100.0%
persona_pattern coverage: 100.0%
few_shot        coverage: 100.0%
spec_driven     coverage: 100.0%

github.com/emilybache/gildedrose-refactoring-kata/gildedrose/gildedrose.go:8: UpdateQuality 100.0%
total: (statements) 100.0%

Abrindo coverage.html (gildedrose.go com as linhas cobertas em verde)...

Fim do roteiro.
```

Achado crítico - Conjured não foi testado

Nenhuma das 5 suítes menciona Conjured. A omissão também está na implementação, não há ramo if para esse tipo em gildedrose.go.

Cenário	Esperado (spec)	Atual (código)
SellIn=3, Quality=6	SellIn=2, Quality=4	SellIn=2, Quality=5
SellIn=0, Quality=6	SellIn=-1, Quality=2	SellIn=-1, Quality=4
SellIn=0, Quality=3	SellIn=-1, Quality=0	SellIn=-1, Quality=1

Regra esperada (especificação canônica): antes da venda, qualidade cai 2/dia; após expirar, cai 4/dia (o dobro de um item normal expirado); nunca abaixo de 0; SellIn continua caindo 1/dia.

Até o Spec-driven, 23 casos, cobertura minuciosa dos 4 tipos conhecidos, não incluiu Conjured.

Outros achados da auditoria

Oráculo derivado do código

Os prompts pediram que a IA deduzisse regras do próprio Código, que já omitia Conjured. Os testes herdaram a mesma omissão.

“Persona é definitiva” é forte demais

A suíte mais completa entre as 4 nativas, mas também omite Conjured. O ranking se mantém; a conclusão precisa de ressalva.

Sulfuras=100 legitima estado inválido

Chain-of-Thought exige que Quality=100 seja preservado; a spec define Sulfuras com qualidade fixa em 80.

Faltam trajetórias multi-dia

Quase todas as suítes testam uma única chamada de UpdateQuality, nenhuma simula vários dias passando.

Robustez de ponteiros não coberta

Persona testa slice nil/vazio, mas nenhuma suíte testa item nil dentro de um slice não vazio.

Veredito final

O falso positivo: 100% de cobertura de statement, sem cobertura de 100% das regras de negócio.

O que funcionou bem

- ✓ 131 testes, 0 FAIL, 100% de cobertura de statement
- ✓ Itens normais, Aged Brie, Sulfuras e Backstage bem cobertos
- ✓ Persona Pattern testa slice nil e vazio
- ✓ Múltiplos itens de tipos diferentes na mesma chamada

A falha crítica

- X Conjured não implementado nem testado por nenhuma técnica
- X Oráculo derivado do código, não da especificação original
- X Erro de implementação torna-se invisível para o teste

Causa raiz: pedir à IA que derive o oráculo do próprio código-fonte, em vez da especificação, é um risco estrutural.

Lições e recomendações

- Cobertura de statement não mede qualidade de teste
- A técnica de prompt continua importando mas para outro tipo de erro

- Derivar o oráculo do próprio código mascara erros de implementação
- Few-shot pode priorizar o formato do exemplo sobre o contexto real

Recomendação final

Manter os testes manuais de Conjured vermelhos como evidência da auditoria até a equipe implementar a regra em produção. Depois da correção, esses mesmos testes tornam-se regressão permanente.

Conclusão: a IA acelera a escrita de testes e antecipa casos de borda, mas não substitui verificação humana: execução real e validação contra a especificação original.



Obrigado!

AI Quality - O Desafio Gilded Rose